2026-05-05-p1-f03-tool-result-persistence-design

P1-F03 — Tool Result Persistence — Design Spec

Date: 2026-05-05 **Author:** Claude Opus 4.7 (1M context) + user (milos85vasic.2nd@gmail.com) **Phase / Feature:** Phase 1, Feature 3 of `docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md` **Status:** APPROVED in brainstorming, awaiting user review of written spec **Successor:** to be handed to `superpowers:writing-plans` for executable plan **Predecessor:** Feature 2 (Permission Rule System) — closed 2026-05-05 (commits `f9e97ff…3fdcb39`)

---

1. Goals, non-goals, success criteria

1.1 What we're building

A claude-code-style tool-result persistence layer for HelixCode: when a tool produces output exceeding 50,000 characters, the runtime saves the raw content to `<cwd>/.helix/tool-results/` and substitutes a path-reference into the message that goes to the LLM. The LLM gets `persistedOutputPath` + `persistedOutputSize` fields plus a system-prompt note instructing it to use the existing `Read` tool when full content is needed. A 7-day age-based sweep runs lazily at startup.

This feature exists because LLM context windows are finite and tool outputs (build logs, test reports, large file listings, find/grep results) routinely blow past 50K characters. Without persistence, a single chatty tool call can poison a session's remaining token budget.

1.2 Goals (priority order)

- **G1 — No bluff.** A passing test/Challenge guarantees end-user usability per Constitution Article XI §11.9. Specifically: a tool that produces a >50K-char output must produce a real file on disk that the user (or LLM via Read) can subsequently fetch — verified by `os.Stat` and content comparison, not just by inspecting a returned struct.
- **G2 — Extend, don't parallelise.** Same lesson as F01/F02. The new sub-package `internal/tools/persistence/` lives next to `internal/tools/permissions/` and `internal/tools/confirmation/` — same logical layer, different concern. Existing `internal/persistence/` (which manages full-state snapshots of session/memory/focus/template managers) is left untouched: blob storage and state snapshots are different shapes.
- **G3 — Boundary at the LLM provider, not the tool registry.** The threshold check fires when a tool result is about to be serialised into a message that consumes LLM tokens. `internal/tools/registry.Execute` is unchanged. Non-LLM callers (test harnesses, future programmatic users) get raw outputs without unnecessary disk I/O.

- **G4 — Read-back via existing Read tool.** No new tool added to the registry. The LLM follows the path in `persistedOutputPath` using its existing Read capability. A system-prompt note tells the LLM the convention.
- **G5 — Project-scoped + lazy 7-day cleanup.** Blobs land under `<cwd>/.helix/tool-results/` and persist across sessions, matching claude-code semantics. `CleanupOld(7*24h)` runs once per CLI startup in a goroutine — never blocks the user, never adds a long-running background goroutine that needs lifecycle management.

1.3 Non-goals (explicit out-of-scope for F03)

- **N1.** Compression on disk (gzip). Adds CPU cost; defer until observed disk pressure.
- **N2.** Distributed / S3 / network-backed persistence. Not relevant to local CLI use.
- **N3.** Tool-side opt-in/opt-out. Every result over the threshold persists; tools whose outputs are reliably small are no-ops at the threshold check.
- **N4.** Per-tool threshold overrides. Single global `PersistThreshold = 50_000`. Per-tool customisation defers until requested.
- **N5.** Cleanup beyond age-based. No quota-based eviction (LRU, etc.). 7-day window keeps disk use bounded for typical workflows.
- **N6.** Re-using `internal/persistence/Store`. That package's `Store` is a state-snapshot framework with `session.Manager`/`memory.Manager`/etc. wired in — wrong shape for blob storage. Building atop it would force F03 to participate in the snapshot lifecycle, which is gratuitous coupling.
- **N7.** Fixing the F02 dead struct field (`c.permissionsEngine`). Tracked separately in PROGRESS.md parking lot.

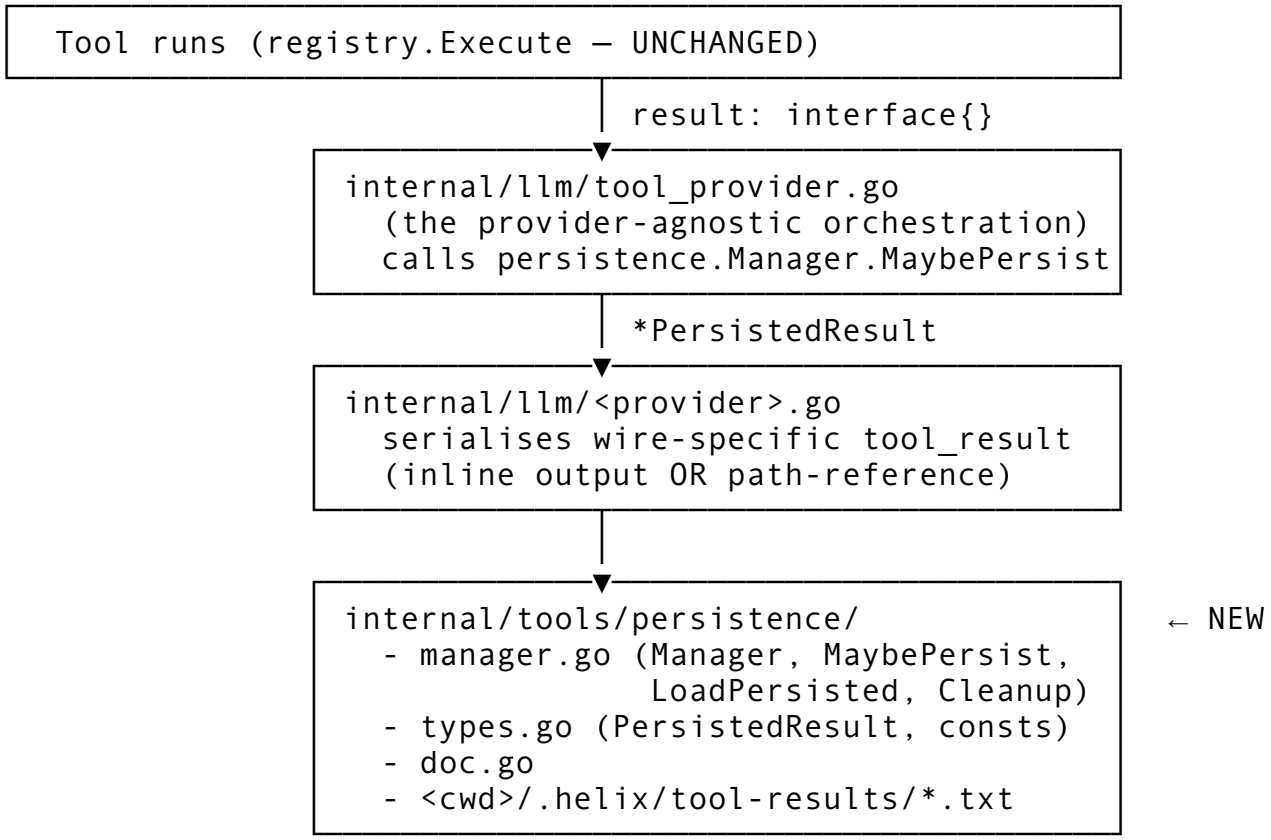
1.4 Success criteria

- **S1.** `make verify-compile` exits 0 with the new package and provider wiring.
- **S2.** Unit tests for `internal/tools/persistence/` pass with `-race`.
- **S3.** Integration test (`-tags=integration`, no mocks) demonstrates: a tool returning >50K chars triggers persistence, the resulting tool_result message contains `persistedOutputPath` (NOT the full content), the file at that path matches the original byte-for-byte, and a subsequent `Read` of that path returns the full content.
- **S4.** Challenge under `tests/e2e/challenges/persistence/` runs an end-to-end scenario with runtime evidence pasted into the close-out commit body. The Challenge proves: file exists, content matches, message-payload omits inline content over the threshold.
- **S5.** Anti-bluff smoke (`grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/tools/persistence/`) returns zero hits.
- **S6.** `CleanupOld(7*24h)` removes only files with `mtime` older than the cutoff. Files inside the directory but with `.gitkeep` or unrelated names are left alone (the cleanup matches its own filename pattern).
- **S7.** Path-traversal guard in `LoadPersisted` rejects any input outside the configured base directory, including `../`-traversal attempts and absolute paths to `/etc/passwd`-style targets.

---

2. Architecture

2.1 Topology



2.2 Why this shape

- **One new sub-package**, mirroring F02. No coupling with the heavyweight `internal/persistence/Store`.
- **Boundary at** `tool_provider.go` means the threshold check happens once, regardless of which LLM provider is in use. Provider-specific wire serialisation reads `*PersistedResult` and renders it into the provider's tool_result schema.
- **Auditable wiring path.** T07 audits every provider that handles `tool_result` to confirm it goes through `tool_provider.go`. Any non-conforming provider gets a direct `MaybePersist` call — the audit produces a list, not a guess.
- **Read-back via existing tool.** The `Read` tool already validates paths and reads files. The system-prompt note teaches the LLM the convention; no new tool surface.

2.3 Component responsibilities

Component	Responsibility
`types.go`	`PersistedResult` struct, constants (`PersistThreshold = 50_000`, `PersistDir = ".helix/tool-results"`, `DefaultMaxAge = 7 * 24 * time.Hour`)
`manager.go`	`Manager` struct (project-rooted), `NewManager(projectRoot)`,

Component	Responsibility
	`MaybePersist(toolName, callID, output) → *PersistedResult, LoadPersisted(path) → string` (path-traversal guard), `CleanupOld(maxAge) error`
`internal/llm/tool_provider.go` (extended)	Receives `*Manager` via constructor injection; wraps every tool result through `MaybePersist` before populating the next request payload
`internal/llm/anthropic_provider.go` (audited)	Reads `PersistedResult.Output` (inline) or `PersistedResult.PersistedOutputPath` (reference) and renders into Anthropic tool_result content blocks
Other providers (`openai_*`, `azure_*`, `ollama_*`, `bedrock_*`)	Same audit. Each gets MaybePersist either through `tool_provider.go` or directly.
`cmd/cli/main.go` (extended)	Construct `persistence.Manager` once at startup with `os.Getwd()`. Trigger lazy `go manager.CleanupOld(DefaultMaxAge)` in a goroutine. Pass `*Manager` to provider constructors.
System prompt template	Add a one-paragraph note: "Tool outputs over 50,000 characters are saved to disk. When you receive a `persistedOutputPath`, use the Read tool with that path to fetch the full content."

3. Data shapes

3.1 Constants

```go
// internal/tools/persistence/types.go
package persistence

import "time"

const (
    PersistThreshold = 50_000
    PersistDir       = ".helix/tool-results"
    DefaultMaxAge    = 7 * 24 * time.Hour
)
```

3.2 `PersistedResult`

```go
type PersistedResult struct {
    Output              string
        `json:"output,omitempty"`              // empty if
        persisted
```

```
    PersistedOutputPath string
        `json:"persistedOutputPath,omitempty"` // absolute path
        on disk
    PersistedOutputSize int
        `json:"persistedOutputSize,omitempty"` // chars (=
        len([]byte(content)))
    WasPersisted        bool   `json:"wasPersisted"`
    ToolName            string `json:"toolName"`
    ToolCallID          string `json:"toolCallID,omitempty"`
}
```

`Output` and `PersistedOutputPath` are mutually exclusive. `PersistedOutputSize` is the byte count of the original content (also equal to `len([]byte(output))`). `WasPersisted` is the canonical boolean — providers that serialise this struct to a wire format check `WasPersisted` to decide which branch to render.

3.3 Manager

```
type Manager struct {
    baseDir string       // <projectRoot>/<PersistDir>
    mu      sync.RWMutex // guards file creation; LoadPersisted
        is read-only and uses RLock
}

func NewManager(projectRoot string) *Manager
func (m *Manager) MaybePersist(toolName, toolCallID string,
        output string) (*PersistedResult, error)
func (m *Manager) LoadPersisted(path string) (string,
        error)     // path-traversal guarded
func (m *Manager) CleanupOld(maxAge time.Duration)
        error          // log + continue on per-file errors
```

3.4 Filename pattern

`<sanitized-tool>_<sha256[:16]>_<UTC-yyyymmdd_hhmmss>.txt`

- `sanitized-tool` strips `/`, `\`, spaces, `..`, control characters, and clamps to 32 chars.
- Hash is the first 16 hex chars of `sha256(output)` — makes identical content idempotent across retries (same filename, no duplicate).
- Timestamp is UTC to keep filenames sortable across timezone changes.

---

4. Threshold semantics

`PersistThreshold = 50_000` characters, applied as `len([]byte(output)) > 50_000`. Inputs:

- Strings: direct.

- Non-string `interface{}` results: converted via `fmt.Sprintf("%v", result)` *before* the size check. The conversion is the responsibility of `tool_provider.go` (where it already happens for serialisation); `MaybePersist` receives a string.
- `nil`: returns `&PersistedResult{Output: "", WasPersisted: false, ToolName: t}`.
- `""`: same as nil — empty inline result.

Boundary cases: - `len([]byte(output)) == 50_000` → inline (boundary is strictly greater than). - `len([]byte(output)) == 50_001` → persisted.

The size check uses bytes, not runes. A 50K-rune CJK output is ~150K bytes and gets persisted. This is correct: token budget tracks bytes downstream, not runes.

---

5. Read-back path

The LLM receives a tool_result message containing the structured fields. The provider wire-format renders it (Anthropic example):

```
{
  "type": "tool_result",
  "tool_use_id": "<callID>",
  "content": "[Tool output persisted to <path> — 142533 chars.
        Use Read to fetch full content.]"
}
```

The `content` field is human-readable (not JSON) so the LLM treats it as a regular string rather than parsing structure. The system-prompt note (added in T08) primes the LLM to recognise the convention and call `Read` when needed.

The existing `Read` tool already validates paths against the workspace root and applies its own size handling (line ranges, etc.), so reading a persisted blob is no more dangerous than reading any other file the LLM has access to.

---

6. Cleanup semantics

`CleanupOld(maxAge time.Duration)` walks `<projectRoot>/.helix/tool-results/`, stats each entry, and removes any FILE (not directory) whose `ModTime()` is older than `time.Now().Add(-maxAge)`.

Rules: - Only files matching the persistence filename pattern (`<tool>_<hash16>_<timestamp>.txt`) are eligible. A user-placed `.gitkeep` or `README.md` is left alone. - Directories are not removed. - Per-file errors are logged and skipped (don't fail the whole sweep on one permission denied). - The function returns the first error encountered (for testing) but always finishes the walk.

Lazy startup invocation: `cmd/cli/main.go` spawns `go func() { _ = manager.CleanupOld(persistence.DefaultMaxAge) }()` so the CLI never blocks on cleanup.

---

7. Error handling

Case	Behaviour
Disk full / permission denied during `MaybePersist`	log warning at WARN; return inline (`WasPersisted: false`, `Output: original`). The tool call succeeds; only persistence fails.
`MkdirAll` fails for `.helix/tool-results/`	same fall-back as above
`LoadPersisted` with path outside `baseDir`	return error wrapping `ErrPathTraversal` (NEW sentinel: `var ErrPathTraversal = errors.New("path outside persistence directory")`)
`LoadPersisted` for missing file	error wrapping `os.ErrNotExist`
`CleanupOld` per-file errors	log + continue; return the first error from the walk
`tool_provider.go` receives nil `*Manager`	treat as "persistence disabled"; pass through inline

The fall-back-to-inline policy preserves the user-visible behaviour (the tool call works) at the cost of a fatter LLM payload. This is the right trade — disk failures shouldn't break tool execution.

---

8. Testing strategy (CONST-035 / Article XI §11.9)

Three layers, mirroring F02:

8.1 Unit (`internal/tools/persistence/*_test.go`)

- `MaybePersist` threshold: 49_999 inline; 50_000 inline (boundary); 50_001 persisted.
- Filename sanitisation: `/`, `\`, `..`, spaces, unicode, names >32 chars.
- Hash idempotence: same content produces same filename.
- `LoadPersisted` happy path; rejects `../etc/passwd`; rejects absolute paths outside `baseDir`.
- `CleanupOld`: removes old files; leaves recent files; leaves non-pattern files (`.gitkeep`); leaves directories.
- Disk-full simulation: `t.TempDir()` + `os.Chmod(dir, 0o500)` → returns inline with logged warning.
- `nil` and empty inputs handled correctly.

Mocks ALLOWED at this layer per CLAUDE.md.

8.2 Integration (`tests/integration/persistence/persistence_integration_test.go`, `-tags=integration`)

- Uses real `tool_provider.go` orchestration loop with a fixture `Tool` that emits `strings.Repeat("X", 60_000)`.

- Asserts: `tool_result` payload coming out the other side has `WasPersisted: true`, `PersistedOutputSize == 60_000`, `Output: ""`.
- Asserts: file exists at `PersistedOutputPath`; `os.ReadFile` returns 60_000 X's.
- Asserts: a subsequent invocation of the existing `Read` tool against `PersistedOutputPath` returns the full content.
- **No mocks.** Uses real filesystem, real tool orchestration. (The fixture Tool is a real Tool struct, not a mock.)

8.3 Challenge (`tests/e2e/challenges/persistence/`)

End-to-end scenario: - A `Bash` tool call invoked via `helixcode permissions check` (or a similar dry-run path that goes through `tool_provider.go`) emits >50K chars of output. - Challenge driver intercepts the resulting message payload (via a debug fixture that captures the tool_provider's outbound message) and asserts `WasPersisted: true`. - File exists at the path; `cmp` matches the original output.

Three scenarios in `expected.json`: - **S1** — output of 49,999 chars → inline (`WasPersisted: false`). - **S2** — output of 50,001 chars → persisted; file content matches. - **S3** — same content emitted twice → same filename (hash idempotence verified).

Runtime evidence: stdout transcript + `ls -la .helix/tool-results/` + `wc -c` of the persisted file pasted into commit body.

8.4 Anti-bluff smoke

`grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/tools/persistence/` must be empty. Run on every sub-commit.

8.5 Mutation test (CONST-039)

Temporarily set `PersistThreshold = 0` so every output persists. Re-run the Challenge — S1 should now fail (WasPersisted: true expected to be false). Revert and confirm S1 passes again.

---

9. Sub-task plan

11 tasks (smaller than F02 because no CLI subcommand group + no slash command):

#	Task	Outputs
T01	Bootstrap `06_phase_1_evidence.md` §F03 + advance PROGRESS to F03-active	docs only
T02	`internal/tools/persistence/types.go` + `doc.go` skeleton (constants, PersistedResult struct)	compile-only
T03	`manager.go`: `Manager`, `NewManager`, `MaybePersist` with hash	unit tests pass

#	Task	Outputs
	idempotence + filename sanitisation (TDD)	
T04	`LoadPersisted` with path-traversal guard + `ErrPathTraversal` sentinel (TDD)	unit tests pass
T05	`CleanupOld` with filename-pattern matching + non-fatal per-file errors (TDD)	unit tests pass
T06	Wire into `internal/llm/tool_provider.go` orchestration loop with constructor injection of `*Manager`	compile + smoke test
T07	Audit and wire each LLM provider (`anthropic_*`, `openai_*`, `azure_*`, `ollama_*`, `bedrock_*`) that handles tool_result content. Direct MaybePersist call where the provider bypasses `tool_provider.go`.	compile + per-provider smoke test
T08	Update system prompt template with persisted-output note; unit test for the rendered prompt	unit test passes
T09	`cmd/cli/main.go` constructs `Manager` at startup + lazy `go CleanupOld`; integration test (`-tags=integration`, no mocks)	integration test passes
T10	Challenge under `tests/e2e/challenges/persistence/` with three scenarios from §8.3; runtime evidence pasted in commit body	Challenge PASS in commit
T11	Feature 3 close-out: anti-bluff scan, `make verify-foundation`, push to all four remotes (no force)	PROGRESS flipped to F04

11 sub-commits expected. F04 (Git Worktree Agent Isolation) unblocked when T11 lands.

---

10. Risks and mitigations

Risk	Mitigation
Some LLM providers bypass `tool_provider.go` and assemble tool_result directly	T07 explicitly audits each provider and adds `MaybePersist` where needed. The audit's output (list of providers + their wiring path) goes into the T07 commit message.
Race between active session reading a blob and `CleanupOld` deleting it	The 7-day window is far longer than any session's lifetime; active sessions read blobs within minutes of writing them. The race is mathematically possible but practically zero. If observed: switch to a lock-file or pid-marker scheme in a follow-up.
Path-traversal vulnerability in `LoadPersisted`	T04 has explicit tests for `../etc/passwd`, `/etc/passwd`, symlink-via-base-dir attacks. The guard uses `filepath.Abs` + prefix check; symlinks under the base dir are accepted (matches `Read` tool's existing behaviour).
`interface{}` tool result with unprintable types (functions, channels)	`fmt.Sprintf("%v", result)` produces `"<func ...>"`-style strings. These are below 50K and pass through inline; no persistence triggered. Document in T03 unit tests.
Filename collision under high concurrency	The hash makes identical content idempotent (same filename). Different content with same hash prefix would collide; sha256[:16] makes that effectively impossible. The sanitised tool name is deterministic from the input, so two simultaneous calls with the same content+tool produce the same filename — and `os.WriteFile` overwrites, which is fine.
`cmd/cli/main.go` is large and tangled (per F02 review notes)	T09 only adds a `*Manager` field + initialiser + goroutine spawn. No restructuring.
F02 follow-up (permissions engine not threaded into ConfirmationCoordinator) intersects with F03	Independent. F03 wires `*Manager` into the LLM provider construction path; F02's wiring gap is in the tool execution dispatch path. They share `cmd/cli/main.go` as a touchpoint but don't conflict.

---

11. References

- Synthesis spec: `docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md` §4.1 (Phase 1 charter)
- Porting doc: `docs/improvements/04_main_plan_step_02/kimi_agent_helix_cli_integration_blueprint/porting_claude_code.md` §Feature 3

- Predecessor plan: `docs/superpowers/plans/2026-05-05-p1-f02-permission-rules.md`
- Predecessor spec: `docs/superpowers/specs/2026-05-05-p1-f02-permission-rules-design.md`
- Evidence file (live): `docs/improvements/06_phase_1_evidence.md`
- Existing infrastructure being audited (NOT modified except for wiring):
  - `helix_code/internal/tools/registry.go` — `Execute` returns `(interface{}, error)`; unchanged
  - `helix_code/internal/llm/tool_provider.go` — orchestration loop where `MaybePersist` is called
  - `helix_code/internal/llm/anthropic_provider.go` — tool_result content serialisation
  - `helix_code/internal/persistence/Store` — heavyweight state-snapshot; NOT used as substrate (per N6)
- Constitutional anchors:
  - Article XI §11.9 — Anti-Bluff Forensic Anchor (every PASS carries runtime evidence)
  - CONST-035 — Zero-Bluff Mandate
  - CONST-039 — Challenge System Integrity (mutation testing mandatory)
  - CONST-042 — No-Secret-Leak (N/A; persisted outputs may contain user data but not credentials by convention)
  - CONST-043 — No-Force-Push (close-out commit T11 pushes without force)

End of P1-F03 Tool Result Persistence design spec.